

Penetration Testing Report

Full Name: Ian Joseph Makokha

Program: HCS - Penetration Testing Internship Week-2

Date:

Introduction

This report document hereby describes the proceedings and results of a Black Box security assessment conducted against the **Week 2 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The objective of the assessment was to uncover vulnerabilities in the **Week 2 Labs** and provide a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	Lab 1: Insecure Direct Object References Lab 2: SQL Injection
-------------------------	--

3. Summary

Outlined is a Black Box Application Security assessment for the **Week 2 Labs**.

Total number of Sub-labs: 16 Sub-labs

High	Medium	Low
4	7	5

High - 4 Sub-labs with hard difficulty level

Medium - 7 Sub-labs with medium difficulty level

Low

- 5 Sub-labs with easy difficulty level

1. Insecure Direct Object References Labs

1.1. Give me my amount!!

Reference	Risk Rating
Give me my amount!!	Low
Tools Used	
Manual Method.	
Vulnerability Description	
Insecure direct object references (IDOR) are a type of access control vulnerability that arises when an application uses user-supplied input to access objects directly.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_1/lab_1.php	
Consequences of not Fixing the Issue	
An IDOR vulnerability can lead to serious data breaches, as attackers can access sensitive information like personal details, financial records, or confidential documents belonging to other users.	
Suggested Countermeasures	
Proper input validation: Always validate user input to ensure it is within expected ranges and formats. Implement robust access controls: Ensure that the application checks if a user has the necessary permissions to access a specific object before granting access. Use opaque identifiers: Instead of directly exposing internal object identifiers, use unique tokens or codes that are not easily guessable by attackers. Session management: Properly manage user sessions to ensure only authorized users can access sensitive data.	
References	
https://portswigger.net/web-security/access-control/idor https://www.varonis.com/blog/what-is-idor-insecure-direct-object-reference https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html https://www.imperva.com/learn/application-security/insecure-direct-object-reference-idor/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

User Profile

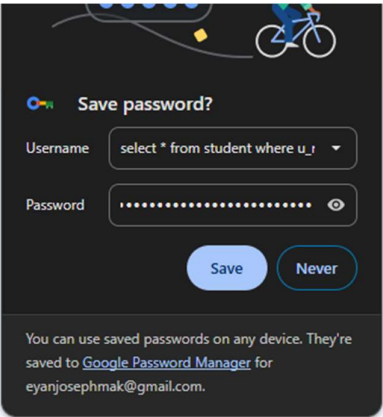
Email

Credit Card

Transaction 1

Transaction 2

Transaction 3



Save password?

Username

Password

You can use saved passwords on any device. They're saved to [Google Password Manager](#) for eyanjosephmak@gmail.com.

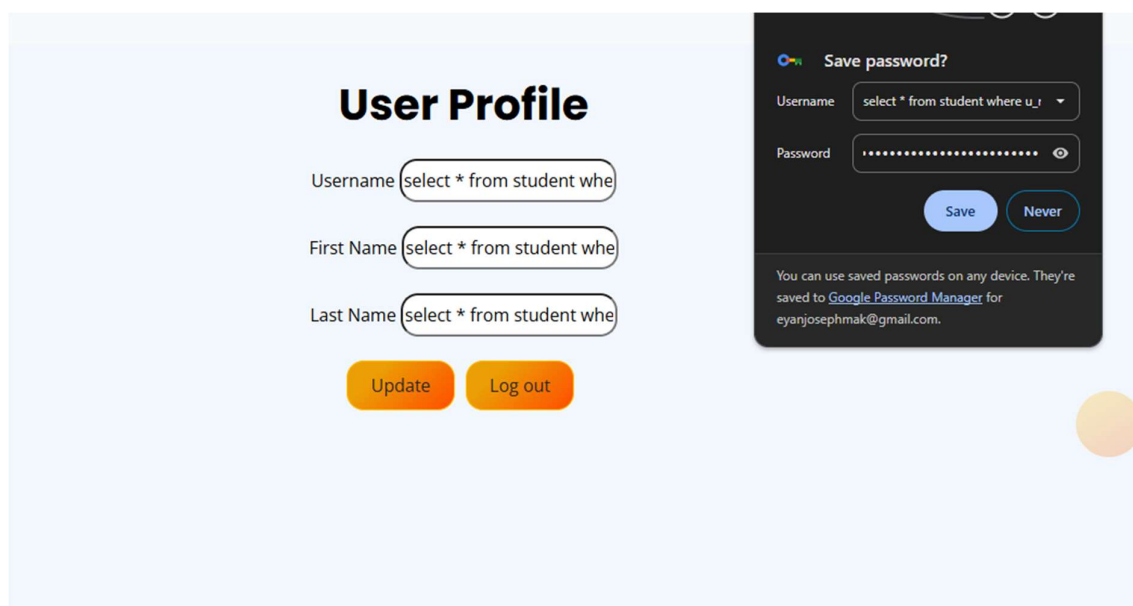
1.2. Stop polluting my params!

Reference	Risk Rating
Stop polluting my params!	Medium
Tools Used	
Manual Method	
Vulnerability Description	
Insecure direct object references (IDOR) are a type of access control vulnerability that arises when an application uses user-supplied input to access objects directly.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_2/lab_2.php	
Consequences of not Fixing the Issue	
An IDOR vulnerability can lead to serious data breaches, as attackers can access sensitive information like personal details, financial records, or confidential documents belonging to other users.	
Suggested Countermeasures	
Proper input validation: Always validate user input to ensure it is within expected ranges and formats. Implement robust access controls: Ensure that the application checks if a user has the necessary permissions to access a specific object before granting access. Use opaque identifiers: Instead of directly exposing internal object identifiers, use unique tokens or codes that are not easily guessable by attackers. Session management:	

Properly manage user sessions to ensure only authorized users can access sensitive data.
References
https://portswigger.net/web-security/access-control/idor https://www.varonis.com/blog/what-is-idor-insecure-direct-object-reference https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html https://www.imperva.com/learn/application-security/insecure-direct-object-reference-idor/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



1.3. Someone changed my Password 🐱! 🐱!

Reference	Risk Rating
Someone changed my Password 🐱! 🐱!	Medium
Tools Used	
Manual Method.	
Vulnerability Description	
Insecure direct object references (IDOR) are a type of access control vulnerability that arises when an application uses user-supplied input to access objects directly.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_3/lab_3.php	

Consequences of not Fixing the Issue
An IDOR vulnerability can lead to serious data breaches, as attackers can access sensitive information like personal details, financial records, or confidential documents belonging to other users.
Suggested Countermeasures
Proper input validation: Always validate user input to ensure it is within expected ranges and formats. Implement robust access controls: Ensure that the application checks if a user has the necessary permissions to access a specific object before granting access. Use opaque identifiers: Instead of directly exposing internal object identifiers, use unique tokens or codes that are not easily guessable by attackers. Session management: Properly manage user sessions to ensure only authorized users can access sensitive data.
References
https://portswigger.net/web-security/access-control/idor https://www.varonis.com/blog/what-is-idor-insecure-direct-object-reference https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html https://www.imperva.com/learn/application-security/insecure-direct-object-reference-idor/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

User Profile

Username

Email

Name

Change Password

Log out

1.4. Change your methods!

Reference	Risk Rating
Change your methods!	Medium
Tools Used	
Manual Method.	
Vulnerability Description	
Insecure direct object references (IDOR) are a type of access control vulnerability that arises when an application uses user-supplied input to access objects directly.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_4/lab_4.php	
Consequences of not Fixing the Issue	
An IDOR vulnerability can lead to serious data breaches, as attackers can access sensitive information like personal details, financial records, or confidential documents belonging to other users.	
Suggested Countermeasures	

Proper input validation:

Always validate user input to ensure it is within expected ranges and formats.

Implement robust access controls:

Ensure that the application checks if a user has the necessary permissions to access a specific object before granting access.

Use opaque identifiers:

Instead of directly exposing internal object identifiers, use unique tokens or codes that are not easily guessable by attackers.

Session management:

Properly manage user sessions to ensure only authorized users can access sensitive data.

References

<https://portswigger.net/web-security/access-control/idor>

<https://www.varonis.com/blog/what-is-idor-insecure-direct-object-reference>

https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

<https://www.imperva.com/learn/application-security/insecure-direct-object-reference-idor/>

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

User Profile

Username

Email

Name

[Change Password](#)

[Log out](#)

2. SQL Injections

2.1. Strings & Errors Part 1!

Reference	Risk Rating
Strings & Errors Part 1!	Low
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_1/lab_1.php	

Consequences of not Fixing the Issue
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.
Suggested Countermeasures
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. - Train and maintain awareness - Don't trust any user input - Use whitelists, not blacklists - Adopt the latest technologies - Employ verified mechanisms - Scan regularly (with Acunetix)
References
https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://www.w3schools.com/sql/sql_injection.asp https://brightsec.com/blog/sql-injection-attack/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

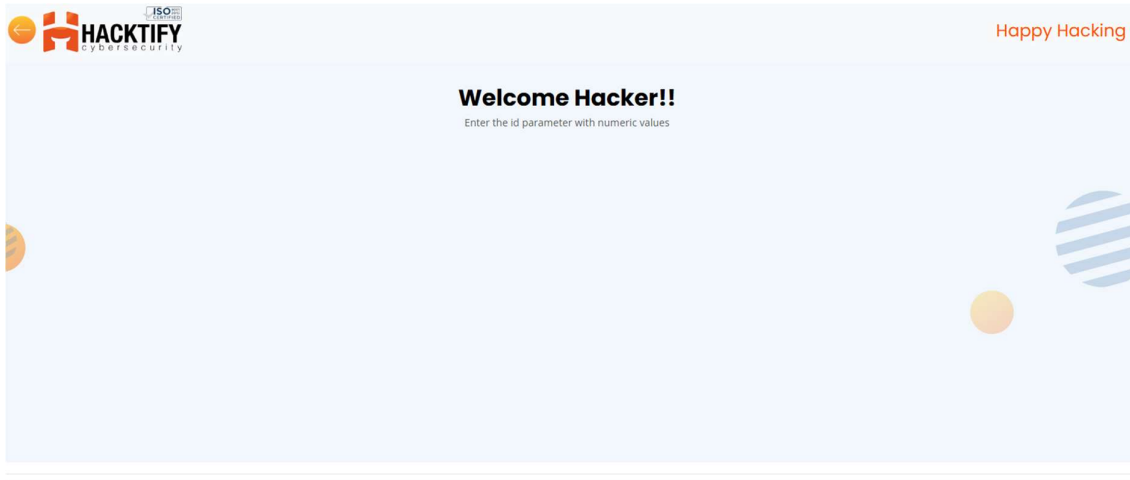
The screenshot displays a web form titled "Admin Login". It features two input fields: "Email:" with a placeholder "Enter Email" and "Password:" with a placeholder "Enter Password". Below these fields is an orange "Login" button. Underneath the button, the text "Email:" and "Password:" is repeated, followed by the message "Successful Login" in a red, bold font.

2.2. Strings & Errors Part 2!

Reference	Risk Rating
Strings & Errors Part 2!	Low
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sql_i_lab/lab_2/lab_2.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	
Suggested Countermeasures	
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. - Train and maintain awareness - Don't trust any user input - Use whitelists, not blacklists - Adopt the latest technologies - Employ verified mechanisms - Scan regularly (with Acunetix)	
References	
https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://www.w3schools.com/sql/sql_injection.asp https://brightsec.com/blog/sql-injection-attack/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



2.3. Strings & Errors Part 3!

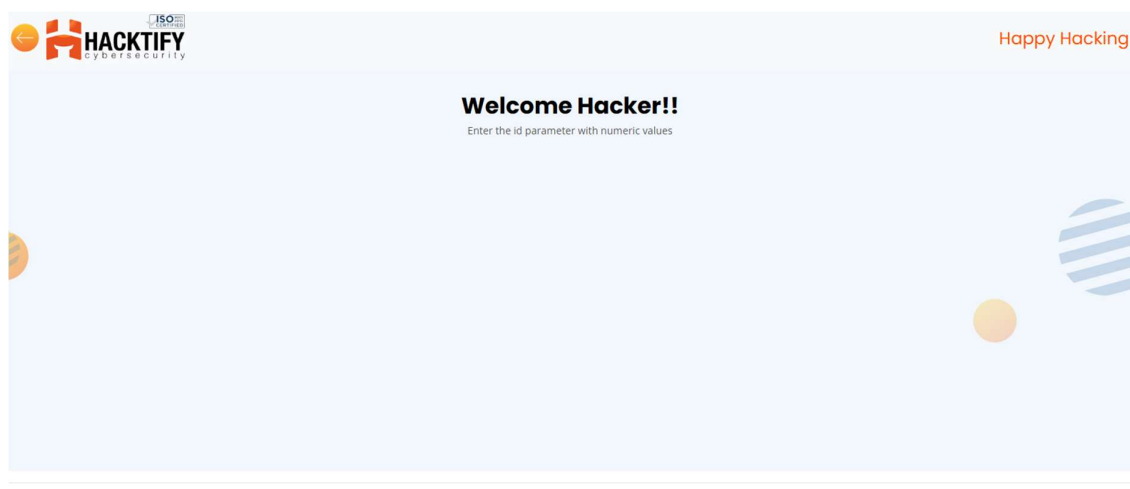
Reference	Risk Rating
Strings & Errors Part 3!	Low
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sql_i_lab/lab_3/lab_3.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	
Suggested Countermeasures	
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. - Train and maintain awareness - Don't trust any user input - Use whitelists, not blacklists - Adopt the latest technologies - Employ verified mechanisms - Scan regularly (with Acunetix)	

References

<https://portswigger.net/web-security/sql-injection>
<https://www.acunetix.com/websitesecurity/sql-injection/>
https://www.w3schools.com/sql/sql_injection.asp
<https://brightsec.com/blog/sql-injection-attack/>

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



2.4. Let's Trick 'em!

Reference	Risk Rating
Let's Trick 'em!	Medium
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sql_i_lab/lab_4/lab_4.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: Passwords. Credit card details.	

Personal user information.

SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.

Suggested Countermeasures

The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements.

Train and maintain awareness

Don't trust any user input

Use whitelists, not blacklists

Adopt the latest technologies

Employ verified mechanisms

Scan regularly (with Acunetix)

References

<https://portswigger.net/web-security/sql-injection>

<https://www.acunetix.com/websitesecurity/sql-injection/>

https://www.w3schools.com/sql/sql_injection.asp

<https://brightsec.com/blog/sql-injection-attack/>

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

Admin Login

Email:

Password:

Login

Query Unsuccessful: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'admin' u_pass='human' '1'='1' AND password='select * from student where u_...' at line 1

2.5. Booleans and Blind!

Reference	Risk Rating
Booleans and Blind!	High
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sql_i_lab/lab_5/lab_5.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: Passwords. Credit card details. Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent	

backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.

Suggested Countermeasures

The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements.

Train and maintain awareness

Don't trust any user input

Use whitelists, not blacklists

Adopt the latest technologies

Employ verified mechanisms

Scan regularly (with Acunetix)

References

<https://portswigger.net/web-security/sql-injection>

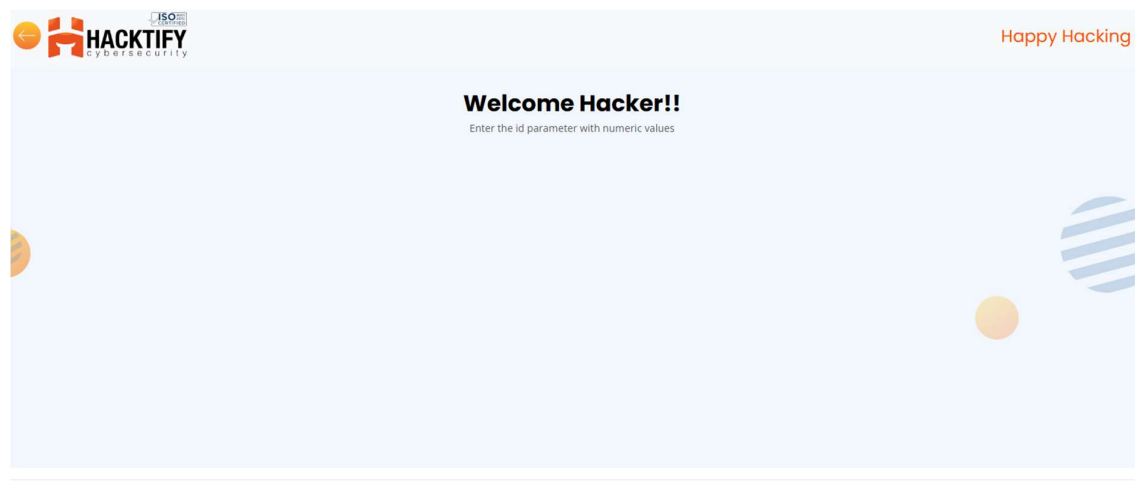
<https://www.acunetix.com/websitesecurity/sql-injection/>

https://www.w3schools.com/sql/sql_injection.asp

<https://brightsec.com/blog/sql-injection-attack/>

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



2.6. Error Based: Tricked

Reference	Risk Rating
Error Based: Tricked	Medium
Tools Used	
Manual Method	
Vulnerability Description	

SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/sql_i_lab/lab_6/lab_6.php
Consequences of not Fixing the Issue
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.
Suggested Countermeasures
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. - Train and maintain awareness - Don't trust any user input - Use whitelists, not blacklists - Adopt the latest technologies - Employ verified mechanisms - Scan regularly (with Acunetix)
References
https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://www.w3schools.com/sql/sql_injection.asp https://brightsec.com/blog/sql-injection-attack/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

Admin Login

Email:

Password:

Login

Incorrect Email and Password

2.7. Errors and Post!

Reference	Risk Rating
Errors and Post!	Low
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_7/lab_7.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	
Suggested Countermeasures	
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements.	

Train and maintain awareness
Don't trust any user input
Use whitelists, not blacklists
Adopt the latest technologies
Employ verified mechanisms
Scan regularly (with Acunetix)

References

<https://portswigger.net/web-security/sql-injection>
<https://www.acunetix.com/websitesecurity/sql-injection/>
https://www.w3schools.com/sql/sql_injection.asp
<https://brightsec.com/blog/sql-injection-attack/>

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

The screenshot shows a web application interface for an 'Admin Login'. It features a title 'Admin Login' in large, bold, black font. Below the title are two input fields: 'Email:' with a placeholder 'Enter Email' and 'Password:' with a placeholder 'Enter Password'. Below these fields is an orange 'Login' button. Below the button, a message is displayed: 'Query Unsuccessful: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'admin' AND u_pass='human' or '1'='1' AND password='select * from student whe...' at line 1'. The entire interface is set against a light blue background.

Admin Login

Email:

Password:

Login

Query Unsuccessful: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'admin' AND u_pass='human' or '1'='1' AND password='select * from student whe...' at line 1

2.8. User Agents lead us!

Reference	Risk Rating
User Agents lead us!	Hard
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sql_i_lab/lab_8/lab_8.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	
Suggested Countermeasures	
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. - Train and maintain awareness - Don't trust any user input - Use whitelists, not blacklists - Adopt the latest technologies - Employ verified mechanisms - Scan regularly (with Acunetix)	
References	
https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://www.w3schools.com/sql/sql_injection.asp https://brightsec.com/blog/sql-injection-attack/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

Admin Login

Email:

Password:

Login

Your IP ADDRESS is: 41.90.180.135

Incorrect Email and Password

2.9. Referrer lead us!

Reference	Risk Rating
Referrer lead us!	Medium
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sql_i_lab/lab_9/lab_9.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	
Suggested Countermeasures	

The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements.

Train and maintain awareness

Don't trust any user input

Use whitelists, not blacklists

Adopt the latest technologies

Employ verified mechanisms

Scan regularly (with Acunetix)

References

<https://portswigger.net/web-security/sql-injection>

<https://www.acunetix.com/websitesecurity/sql-injection/>

https://www.w3schools.com/sql/sql_injection.asp

<https://brightsec.com/blog/sql-injection-attack/>

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

The screenshot displays a web interface for an 'Admin Login'. At the top, the title 'Admin Login' is centered in a large, bold, black font. Below the title, there are two input fields: 'Email:' followed by a text box containing 'Enter Email', and 'Password:' followed by a text box containing 'Enter Password'. Below these fields is an orange, rounded rectangular button with the text 'Login' in black. Underneath the button, the text 'Your IP ADDRESS is: 41.90.180.135' is displayed in a smaller black font. At the bottom of the form, the message 'Incorrect Email and Password' is shown in a bold, orange-red font, indicating a failed login attempt.

2.10. Oh Cookies!

Reference	Risk Rating
Oh Cookies!	Hard
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sql_i_lab/lab_10/lab_10.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	
Suggested Countermeasures	
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. - Train and maintain awareness - Don't trust any user input - Use whitelists, not blacklists - Adopt the latest technologies - Employ verified mechanisms - Scan regularly (with Acunetix)	
References	
https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://www.w3schools.com/sql/sql_injection.asp https://brightsec.com/blog/sql-injection-attack/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

User Login

Username

Enter your username

Password

Enter your Password

Login

Incorrect username and Password

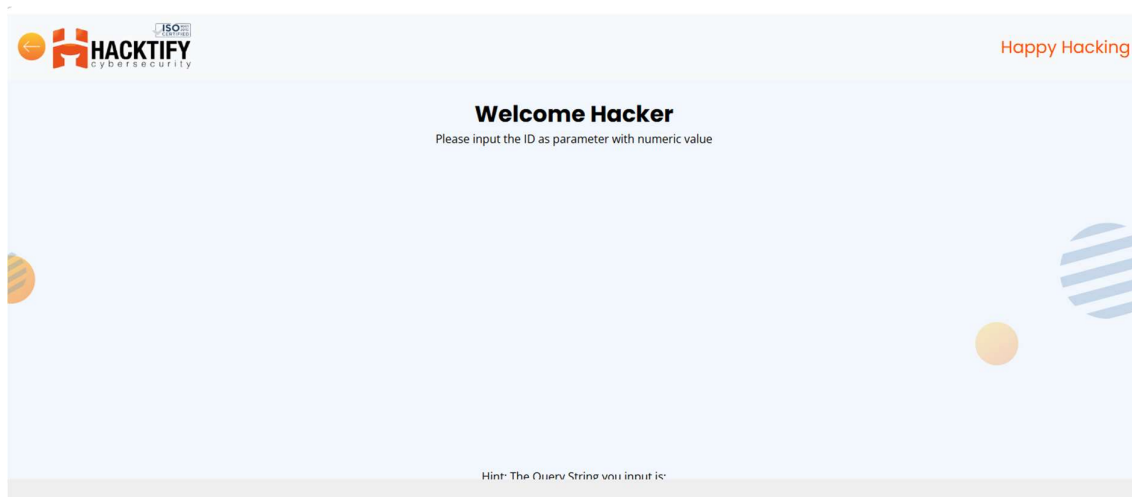
2.11. WAF's are injected!

Reference	Risk Rating
WAF's are injected!	Hard
Tools Used	
Manual Method	
Vulnerability Description	
SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_11/lab_11.php	
Consequences of not Fixing the Issue	
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	

Suggested Countermeasures
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. Train and maintain awareness Don't trust any user input Use whitelists, not blacklists Adopt the latest technologies Employ verified mechanisms Scan regularly (with Acunetix)
References
https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://www.w3schools.com/sql/sql_injection.asp https://brightsec.com/blog/sql-injection-attack/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



2.12. WAF's are injected Part 2!

Reference	Risk Rating
WAF's are injected Part 2!	Medium
Tools Used	
Manual Method	
Vulnerability Description	

SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/sql_i_lab/lab_12/lab_12.php
Consequences of not Fixing the Issue
A successful SQL injection attack can result in unauthorized access to sensitive data, such as: - Passwords. - Credit card details. - Personal user information. SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.
Suggested Countermeasures
The only sure way to prevent SQL Injection attacks is input validation and parametrized queries including prepared statements. - Train and maintain awareness - Don't trust any user input - Use whitelists, not blacklists - Adopt the latest technologies - Employ verified mechanisms - Scan regularly (with Acunetix)
References
https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://www.w3schools.com/sql/sql_injection.asp https://brightsec.com/blog/sql-injection-attack/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

Welcome Hacker

Please input the ID as parameter with numeric value

Hint: The Query String you input is: